Щучкин Максим, Б07-202

Отчет по лабораторной работе #3

«Использование семафоров и общих сегментов памяти»

Цель: написать программу, решающую задачу численного интегрирования с помощью тяжелых и легких процессов, объединённых механизмами синхронизации: семафорами System V для межпроцессного взаимодействия и мьютексами для синхронизации потоков внутри процесса.

Текст программы:

```
#define _GNU_SOURCE
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/ipc.h>
#include <sys/shm.h>
#include <sys/sem.h>
#include <pthread.h>
#include <omp.h>

#define SHM_ID 2008
#define SEM_ID 2007
#define PERMS 0600

/* Структура для общей памяти: сумма и счётчик готовых процессов */
typedef struct { double sum; int ready; } shared_t;

/* Параметры для потока */
typedef struct { int mp, mt, nt, n; double a, b, res; pthread_mutex_t *mut;
double *psum; } tdata_t;

// Интегрируемая функция
double f(double x) { return 4.0 / (1.0 + x * x); }

/* Метод средних прямоугольников */
double integrate(double (*func)(double), double a, double b, int n) {
    double h = (b - a) / n, s = 0.0;
    for (int i = 0; i < n; i++) s += func(a + h * (i + 0.5));
    return h * s;
}

// my_job
void* thread_func(void* arg) {
    tdata_t* td = (tdata_t*)arg;
    double h = (td->b - td->a) / td->nt;
```

```
    double a1 = td->a + h * td->mt;
    double b1 = (td->mt == td->nt - 1) ? td->b : a1 + h;

    td->res = integrate(f, a1, b1, td->n);
    fprintf(stderr, "mp=%d mt=%d a1=%le b1=%le n1=%d s1=%le\n",
            td->mp, td->mt, a1, b1, td->n, td->res);

    pthread_mutex_lock(td->mut);
    *(td->psum) += td->res;
    pthread_mutex_unlock(td->mut);
    return NULL;
}

union semun { int val; struct semid_ds* buf; unsigned short* array; };

int main(int argc, char* argv[]) {
    if (argc < 3) { printf("Usage: %s <np> <nt>\n", argv[0]); return 1; }
    int np = atoi(argv[1]), nt = atoi(argv[2]);
    if (np < 1) np = 1; if (nt < 1) nt = 1;
    setbuf(stderr, NULL);

    double t_start = omp_get_wtime();
    int mp = 0;
    pid_t spid = 0;
    double proc_sum = 0.0;
    pthread_mutex_t mut = PTHREAD_MUTEX_INITIALIZER;

    /* Создание IPC-ресурсов до ветвления */
    int shmid = shmget(SHM_ID, sizeof(shared_t), PERMS | IPC_CREAT);
    int semid = semget(SEM_ID, 1, PERMS | IPC_CREAT);
    if (shmid < 0 || semid < 0) { perror("shm/sem get"); exit(1); }

    shared_t* shm = (shared_t*)shmat(shmid, NULL, 0);
    if (shm == (void*)-1) { perror("shmat"); exit(1); }

    union semun arg; arg.val = 1;
    semctl(semid, 0, SETVAL, arg);
    if (mp == 0) { shm->sum = 0.0; shm->ready = 0; }

    /* Создание тяжелых процессов */
    for (int i = 1; i < np; i++) {
        spid = fork();
        if (spid < 0) { perror("fork"); exit(1); }
        if (spid == 0) { mp = i; break; }
    }

    /* Распределение работы */
    double h_proc = 1.0 / np;
```

```
    double a_proc = h_proc * mp;
    double b_proc = (mp == np - 1) ? 1.0 : a_proc + h_proc;
    int n1 = 1000000000 / (np * nt);

    /* Создание потоков */
    pthread_t* threads = malloc(nt * sizeof(pthread_t));
    tdata_t* targs = malloc(nt * sizeof(tdata_t));
    for (int t = 0; t < nt; t++) {
        targs[t] = (tdata_t){mp, t, nt, n1, a_proc, b_proc, 0.0, &mut,
&proc_sum};
        pthread_create(&threads[t], NULL, thread_func, &targs[t]);
    }
    for (int t = 0; t < nt; t++) pthread_join(threads[t], NULL);
    free(threads); free(targs);

    /* Агрегация через семафор */
    struct sembuf lock = {0, -1, 0}, unlock = {0, +1, 0};
    semop(semid, &lock, 1);
    shm->sum += proc_sum;
    shm->ready++;
    semop(semid, &unlock, 1);

    /* Мастер ждет завершения и выводит результат */
    if (mp == 0) {
        while (shm->ready < np) usleep(100);
        double t_elapsed = omp_get_wtime() - t_start;
        printf("time=%lf sum= %le\n", t_elapsed, shm->sum);

        shmdt(shm);
        shmctl(SHM_ID, IPC_RMID, NULL);
        semctl(SEM_ID, 0, IPC_RMID);
    } else {
        shmdt(shm);
        exit(0);
    }
    return 0;
}
```

Output:

```
fnbik09@sr210: ~/mpc/task3
fnbik09@sr210:~/mpc/task3$ gcc -O2 -fopenmp -pthread solution_03.c -o solution_03.px -lm
fnbik09@sr210:~/mpc/task3$ ./solution_03.px 1 1
mp=0 mt=0 a1=0.000000e+00 b1=1.000000e+00 n1=1000000000 s1=3.141593e+00
time=1.731389 sum= 3.141593e+00
fnbik09@sr210:~/mpc/task3$ ./solution_03.px 2 1
mp=0 mt=0 a1=0.000000e+00 b1=5.000000e-01 n1=500000000 s1=1.854590e+00
mp=1 mt=0 a1=5.000000e-01 b1=1.000000e+00 n1=500000000 s1=1.287002e+00
time=0.876095 sum= 3.141593e+00
fnbik09@sr210:~/mpc/task3$ ./solution_03.px 1 2
mp=0 mt=0 a1=0.000000e+00 b1=5.000000e-01 n1=500000000 s1=1.854590e+00
mp=0 mt=1 a1=5.000000e-01 b1=1.000000e+00 n1=500000000 s1=1.287002e+00
time=0.879435 sum= 3.141593e+00
fnbik09@sr210:~/mpc/task3$ ./solution_03.px 2 2
mp=0 mt=1 a1=2.500000e-01 b1=5.000000e-01 n1=250000000 s1=8.746758e-01
mp=1 mt=1 a1=7.500000e-01 b1=1.000000e+00 n1=250000000 s1=5.675882e-01
mp=0 mt=0 a1=0.000000e+00 b1=2.500000e-01 n1=250000000 s1=9.799147e-01
mp=1 mt=0 a1=5.000000e-01 b1=7.500000e-01 n1=250000000 s1=7.194140e-01
time=0.523507 sum= 3.141593e+00
fnbik09@sr210:~/mpc/task3$
```

Выводы:

1) Во всех запусках получено sum ≈ 3.141593, что совпадает с аналитическим значением π — алгоритм интегрирования и синхронизация работают верно.
2) Время падает с ростом числа рабочих единиц (1.73с → 0.87с → 0.52с), что подтверждает эффективность гибридного распараллеливания для данной задачи.
3) Отсутствие конфликтов и потерь данных при одновременной записи в общую память свидетельствует о корректной работе семафоров.